

# Optimisation d'un trajet à l'aide d'algorithmes

Face123moka@gmail.com



Année : 2023/2024

# Introduction:

- Construction d'un réseau des chemins optimaux
- Détermination des chemins optimaux de ce réseau
- Utilisation de la théorie des graphes pour résoudre le deuxième problème
- La recherche du chemin optimal peut être complexe

# Quel est le chemin le plus court :

## Définition :

Le chemin le plus court est le trajet optimal entre deux points sur un réseau, qui minimise la distance ou le coût nécessaire pour passer d'un point à un autre.

## Applications :

Il est utilisé dans des domaines comme le transport, la communication réseau, la visualisation de données et même dans les jeux vidéo pour créer un monde plus immersif.

## Les types d'algorithmes:

L'algorithme de Dijkstra, BFS et A\* sont des algorithmes courants utilisés pour trouver le chemin le plus court, chacun avec son approche et ses forces uniques.



# PLAN :

- Définitions
- Algorithme et son fonctionnement
  - Parcours en largeur (BFS)
  - Dijkstra
  - A\*
- Application sur une Carte d'une ville
- Annexes



# Définitions:

## Graphe:

Un graphe est une structure mathématique composée de sommets (ou nœuds) reliés entre eux par des arêtes (ou arc), représentant des relations ou des connexions entre les entités.

## Arc :

- Un arc est une liaison unidirectionnelle entre deux sommets, indiquant une relation avec un sens spécifique
- Une arête peut être bidirectionnelle

## Graphe pondéré:

Un graphe (orienté ou non-orienté) est dit pondéré si chaque arête (arc) possède un poids ( un nombre réel en general) .

## Sommets adjacents:

- Deux sommets adjacents lorsqu'ils sont reliés par un arc.
- Un sommet qui ne possède pas un sommet adjacent est dit isolé.

## Chemin:

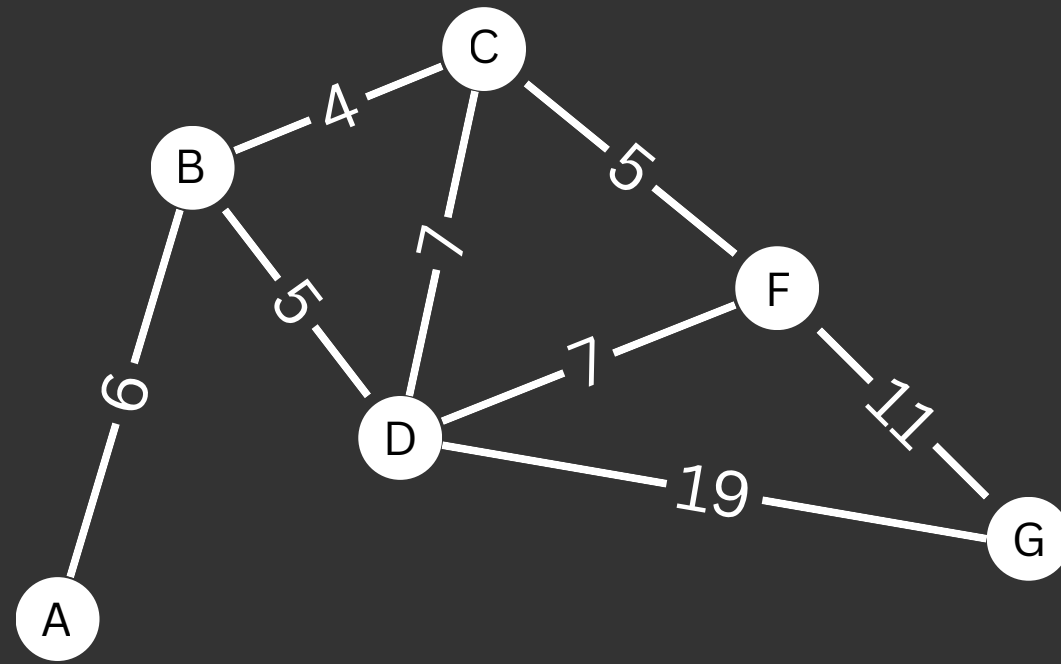
Une suite  $(a,b,c,\dots)$  est dite chemin si chaque paire  $(a,b)$ ,  $(b,c)$ ,  $(c,\dots)$  est un arc.

## Matrice associée à un graphe

On numérote les sommets d'un graphe  $(G)$  d'ordre  $n$ . La matrice associée au graphe  $(G)$  est la matrice carrée d'ordre  $n$  où le terme  $a_{ij}$  situé à l'intersection de la ligne  $i$  et la colonne  $j$ , est égale au nombre d'arête reliant les sommets  $i$  et  $j$ .

## Ordre d'un graphe:

L'ordre d'un graphe est le nombre total de ses sommets



**G un graphe pondéré non-orienté**

### Chemins entre A et G :

valeur du chemin A-B-D-C-F-G : 34

valeur du chemin A-B-C-D-F-G : 35

valeur du chemin A-B-C-F-G : 26 —→ le chemin optimal

valeur du chemin A-B-D-F-G : 29

valeur du chemin A-B-C-D-G : 36

valeur du chemin A-B-D-G : 30

$$M = \begin{pmatrix} 0 & 6 & \infty & \infty & \infty & \infty \\ 6 & 0 & 4 & 5 & \infty & \infty \\ \infty & 4 & 0 & 7 & 5 & \infty \\ \infty & 5 & 7 & 0 & 7 & \infty \\ \infty & \infty & 5 & 7 & 0 & 11 \\ \infty & \infty & \infty & \infty & 11 & 0 \end{pmatrix}$$

A  
B  
C  
D  
F  
G

**M la matrice associée à G**

# Algorithme et son fonctionnement :

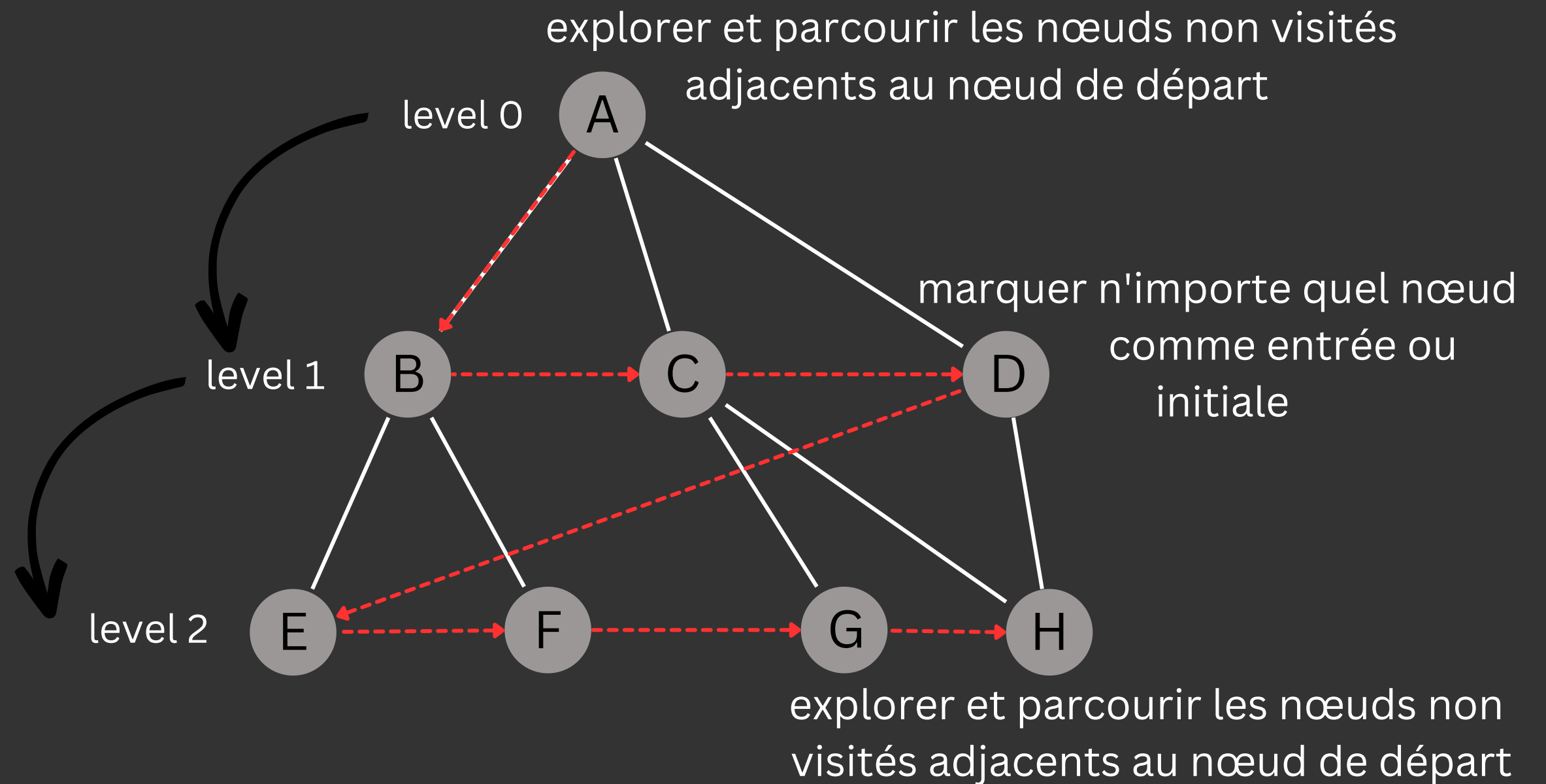
- BFS
- Dijkstra
- $A^*$

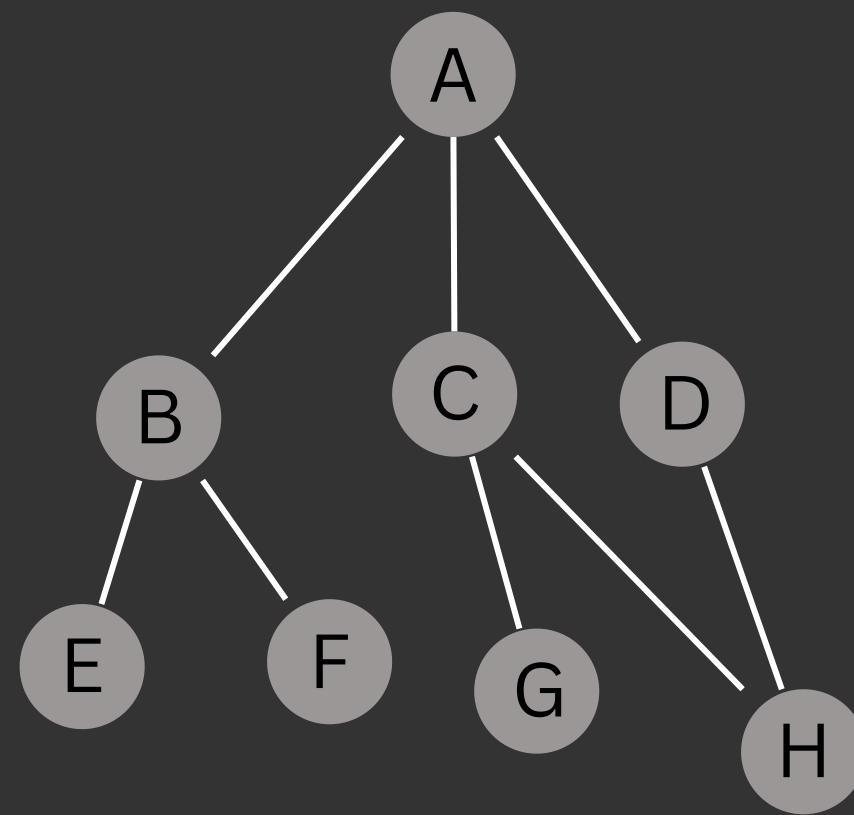


# Algorithme de parcours en largeur (BFS) :

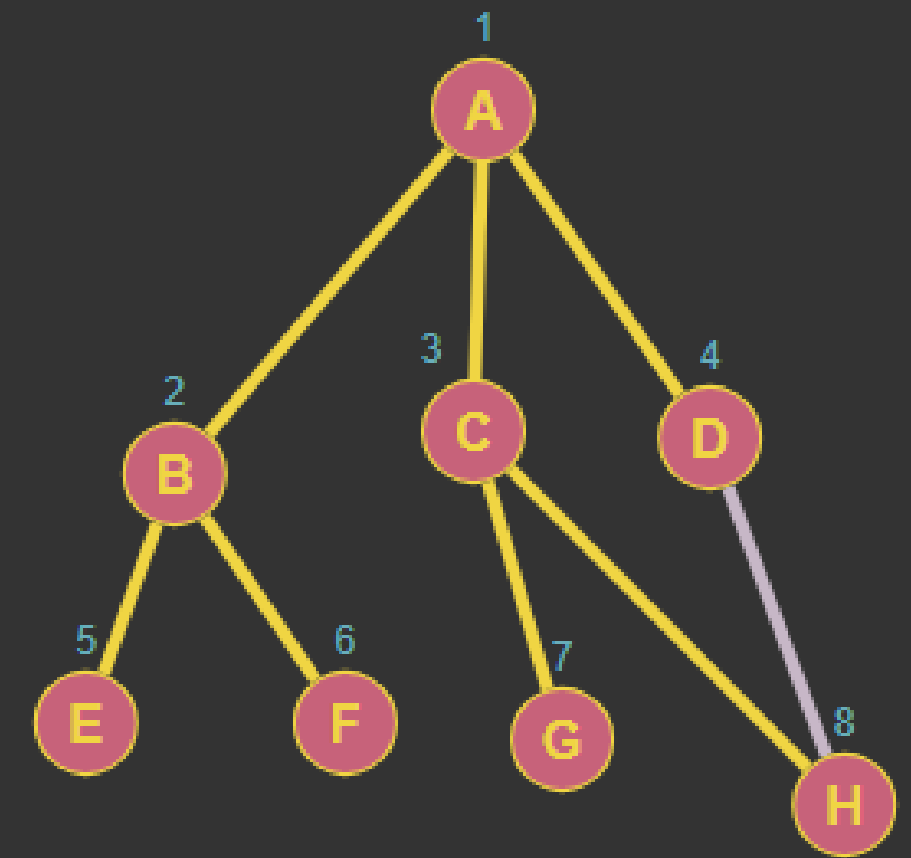
## Définition :

Le BFS est un algorithme de recherche qui explore les nœuds d'un graphe de manière itérative, en utilisant une file d'attente pour gérer l'ordre de visite.





BFS



BFS(G,s)

Données : graphe G, sommet de départ s

File Q (initialisée à vide), marque des sommets (initialisé à Faux)

**début**

marque[s] ← Vrai ;

enfiler s à la fin de Q;

**tant que** Q non vide **faire**

u ← tête(Q) ;

**pour** chaque v voisin de u **faire**

**si** v non marqué **alors**

marque[v] ← Vrai ;

enfiler v à la fin de Q;

**fin**

**fin**

défiler u de la tête de Q;

**fin**

**fin**

output

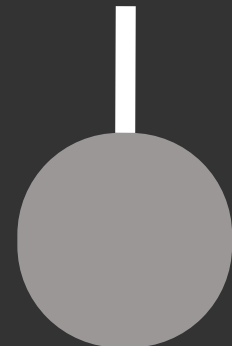
| FILE         |              |              |              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| A            |              |              |              |              |              |              |              |
| <del>A</del> | B            | C            | D            |              |              |              |              |
| <del>A</del> | <del>B</del> | C            | D            | E            | F            |              |              |
| <del>A</del> | <del>B</del> | <del>C</del> | D            | E            | F            | G            | H            |
| <del>A</del> | <del>B</del> | <del>C</del> | <del>D</del> | E            | F            | G            | H            |
| <del>A</del> | <del>B</del> | <del>C</del> | <del>D</del> | <del>E</del> | F            | G            | H            |
| <del>A</del> | <del>B</del> | <del>C</del> | <del>D</del> | <del>E</del> | <del>F</del> | G            | H            |
| <del>A</del> | <del>B</del> | <del>C</del> | <del>D</del> | <del>E</del> | <del>F</del> | <del>G</del> | H            |
| <del>A</del> | <del>B</del> | <del>C</del> | <del>D</del> | <del>E</del> | <del>F</del> | <del>G</del> | <del>H</del> |

# Algorithme de Dijkstra :

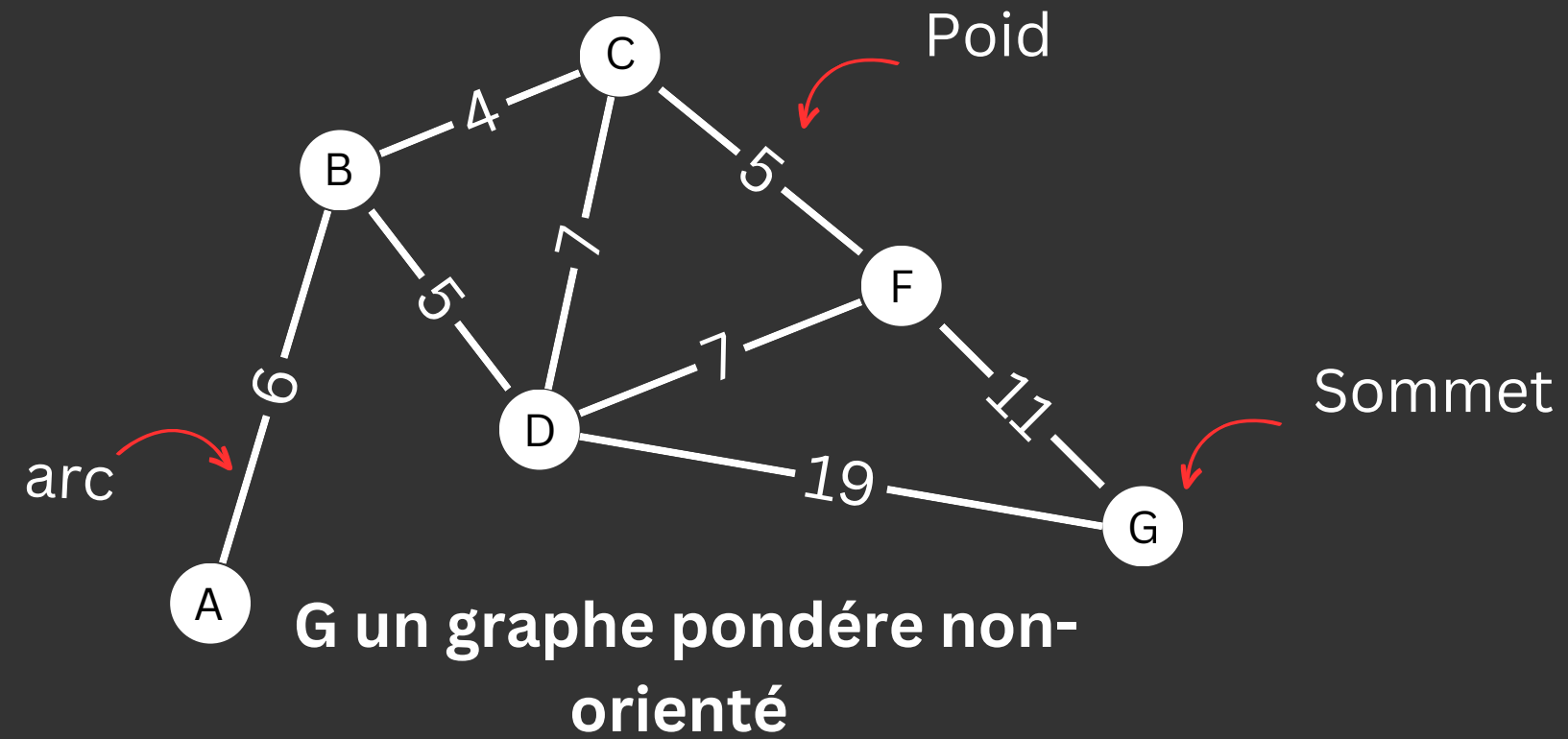


Edsger Dijkstra

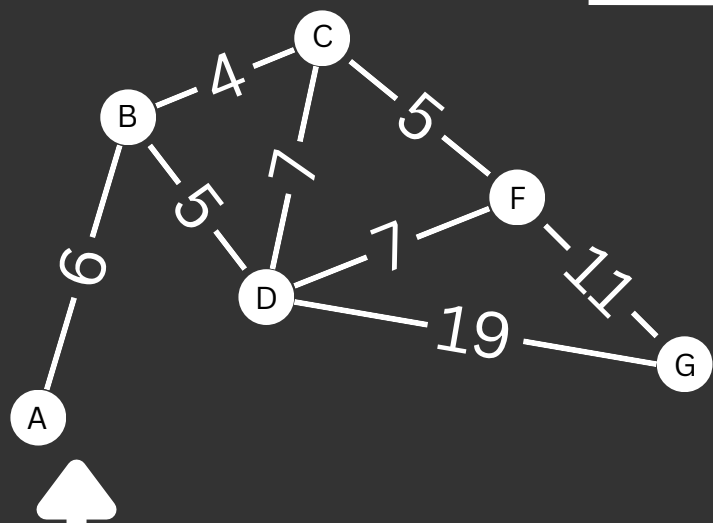
1959



Qui?

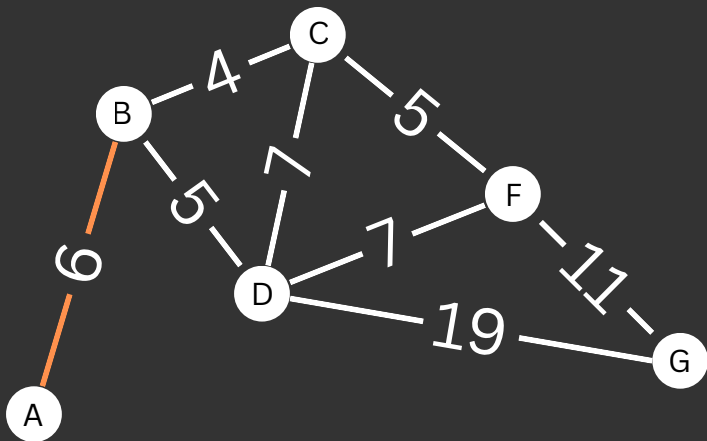


Comment ça marche?



P=[A]

| Sommet | A | B   | C         | D         | F         | G         |
|--------|---|-----|-----------|-----------|-----------|-----------|
| A      | 0 | 6-A | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |



Dijkstra(G,s)

Données : graphe G, sommet de départ s

début

Créer une liste P **vide**

Créer un tableau T de taille égale au nombre de sommets de G

$d[s] = 0$

$d[a] = +\infty$  pour chaque sommet a

**Tant que** la liste P ne contient pas tous les sommets

    Choisir un sommet u hors de P de plus petite distance  $d[u]$ ;

    Ajouter u à la liste P;

**Pour** chaque v hors de P adjacent à u **faire**

**Si**  $d[v] > d[u] + \text{poids}(u,v)$

$d[v] = d[u] + \text{poids}(u,v)$ ;

**Fin si**

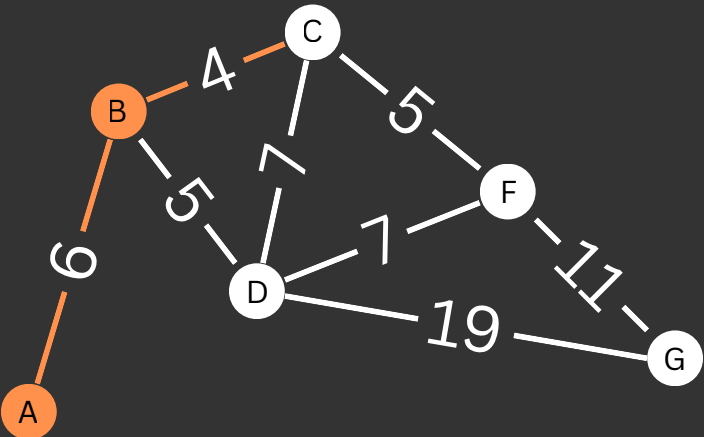
**Fin pour**

**Fin tant que**

**Fin**

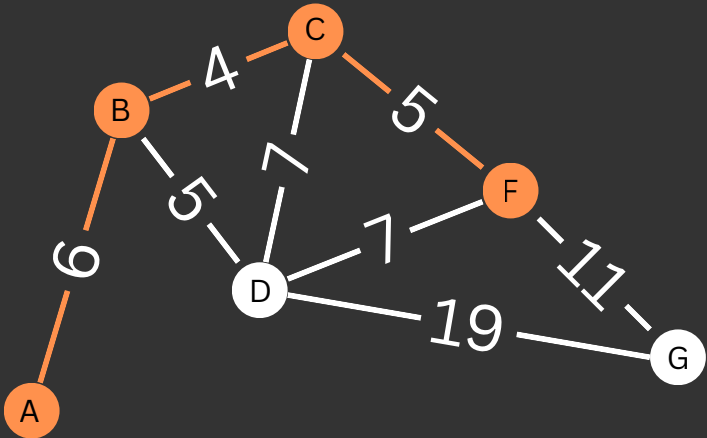
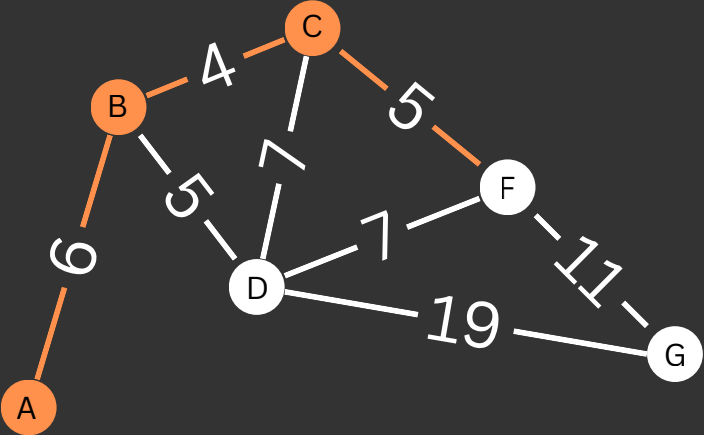
P=[A,B]

| Sommet | A | B   | C         | D         | F         | G         |
|--------|---|-----|-----------|-----------|-----------|-----------|
| A      | 0 | 6-A | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| B      |   | 6-A | 10-B      | 11-B      | $+\infty$ | $+\infty$ |



P=[A,B,C]

| Sommet | A | B   | C         | D         | F         | G         |
|--------|---|-----|-----------|-----------|-----------|-----------|
| A      | 0 | 6-A | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| B      |   | 6-A | 10-B      | 11-B      | $+\infty$ | $+\infty$ |
| C      |   |     | 10-B      | 11-B      | 15-C      | $+\infty$ |



P=[A,B,C,D]

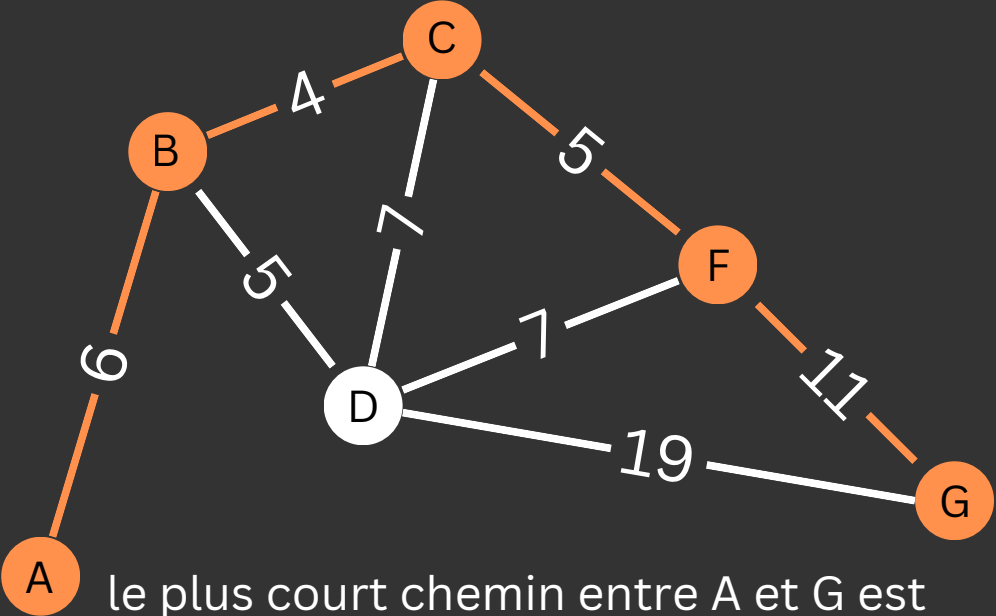


| Sommet | A | B   | C         | D         | F         | G         |
|--------|---|-----|-----------|-----------|-----------|-----------|
| A      | 0 | 6-A | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| B      |   | 6-A | 10-B      | 11-B      | $+\infty$ | $+\infty$ |
| C      |   |     | 10-B      | 11-B      | 15-C      | $+\infty$ |
| D      |   |     |           | 11-B      | 15-C      | 30-D      |



P=[A,B,C,D,F,G]

| Sommet | A | B   | C         | D         | F         | G         |
|--------|---|-----|-----------|-----------|-----------|-----------|
| A      | 0 | 6,A | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| B      |   | 6-A | 10-B      | 11-B      | $+\infty$ | $+\infty$ |
| C      |   |     | 10-B      | 11-B      | 15-C      | $+\infty$ |
| D      |   |     |           | 11-B      | 15-C      | 30-D      |
| F      |   |     |           |           | 15-C      | 26-F      |
| G      |   |     |           |           |           | 26-F      |



le plus court chemin entre A et G est  
A-B-C-F-G de cout : 26



# Algorithme A\* :

- Il commence comme l'algorithme de Dijkstra mais utilise une fonction heuristique supplémentaire pour estimer la distance à parcourir pour chaque voisin possible
- A\* utilise une fonction heuristique  $f = g + h$  dans la file de priorité.
- Pour retourner une solution optimale, l'algorithme A\* doit

utiliser une **fonction heuristique admissible** :

La fonction  $h$  ne doit jamais surestimer la distance du but.

# Algorithme A\*

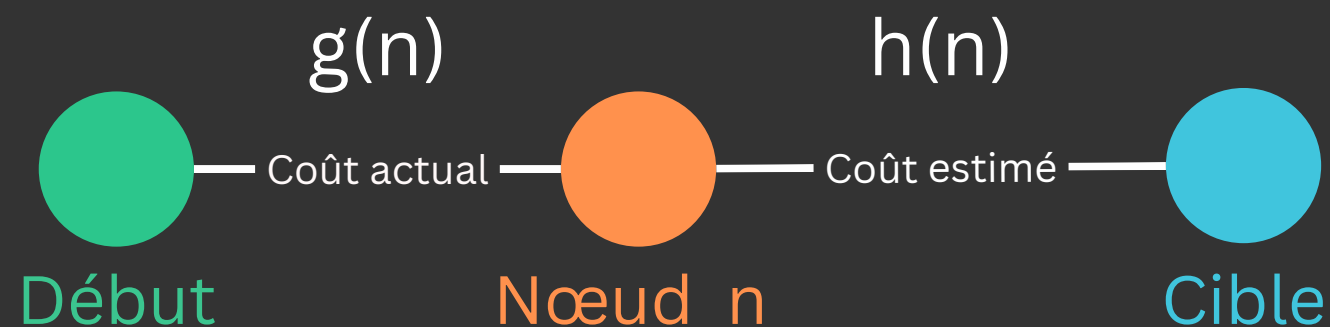
VS

# Dijkstra

- Trouver le chemin le plus court du nœud de départ au **nœud cible** dans le graphe

$$f(n) = g(n) + h(n)$$

Information sur le nœud cible



- Ne pas étendre tous les nœuds

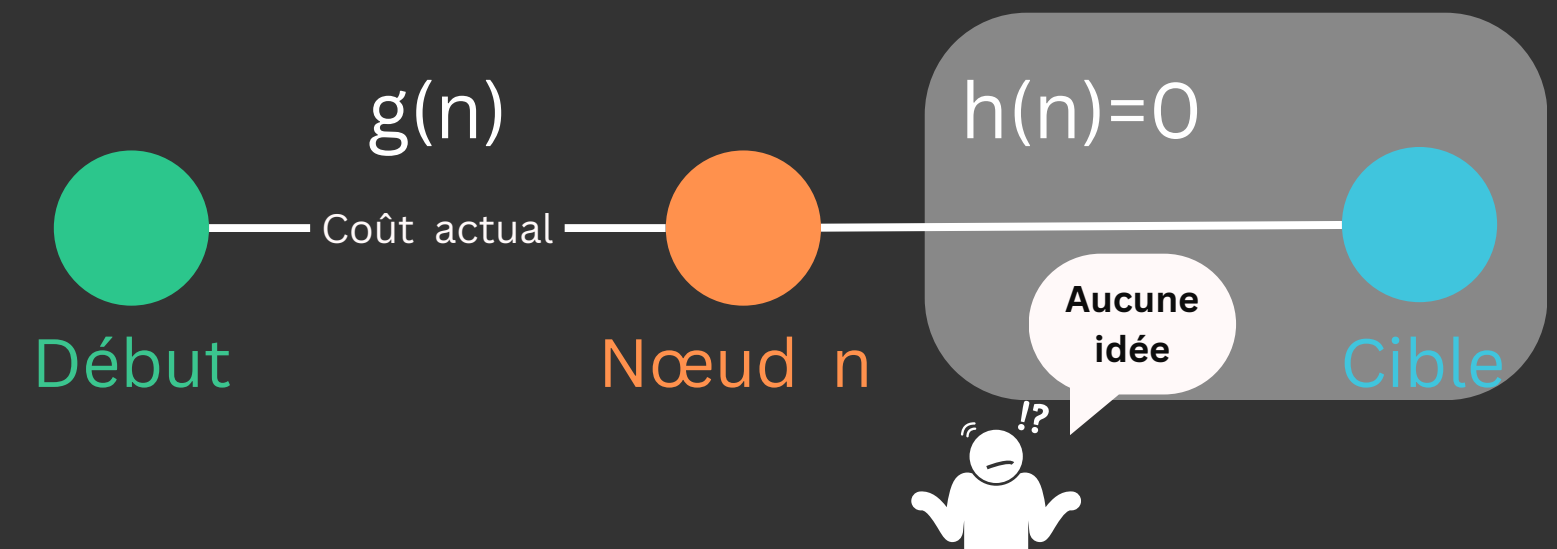
diriger le chemin en utilisant une heuristique

- Trouver le chemin le plus court du nœud de départ à tous **les autres nœuds** du graphe

$$f(n) = g(n)$$

$h(n) = 0$

Aucune information sur le nœud cible

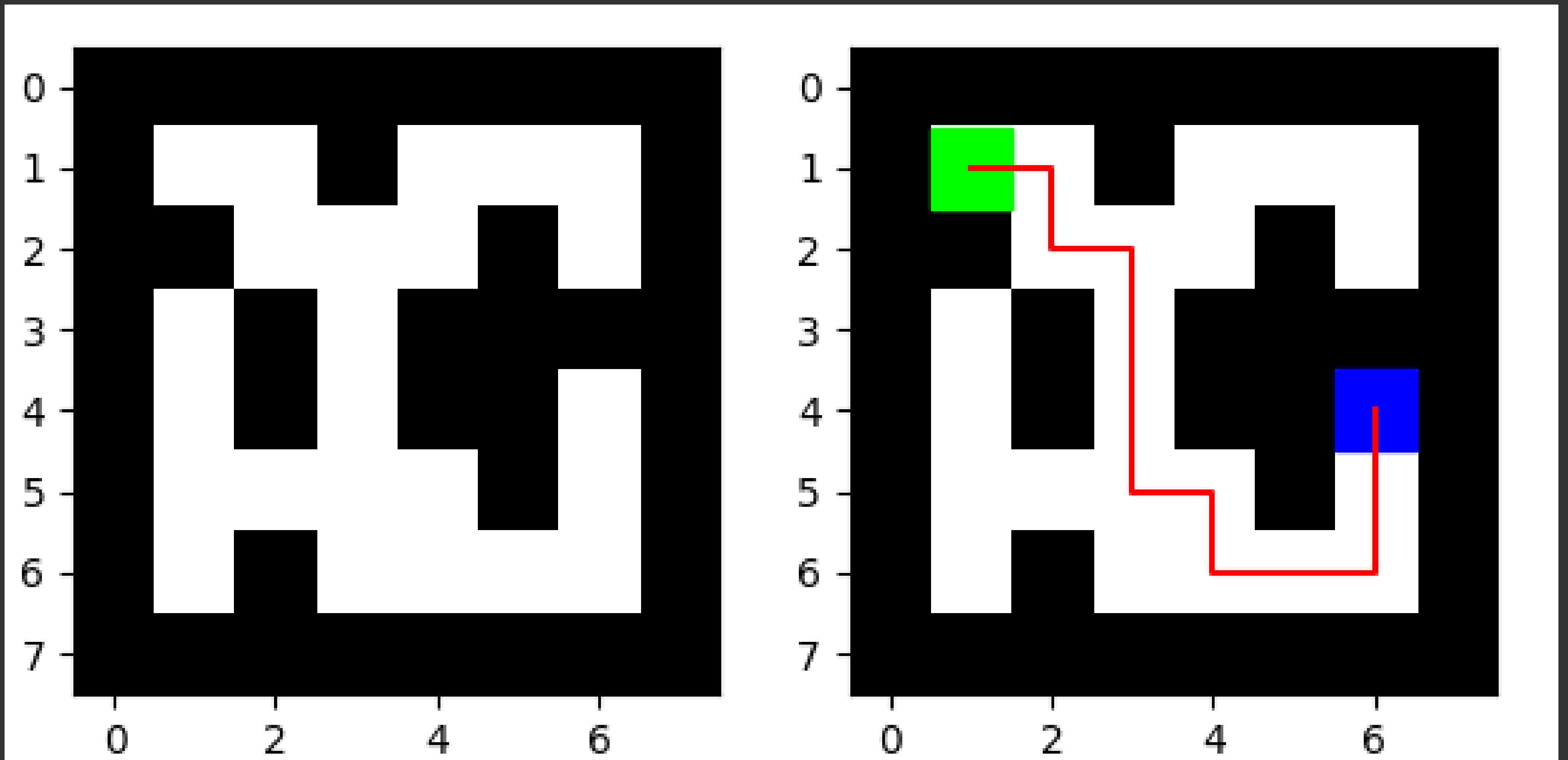


- Étendre tous les nœuds

**Application sur une Carte d'une ville :**

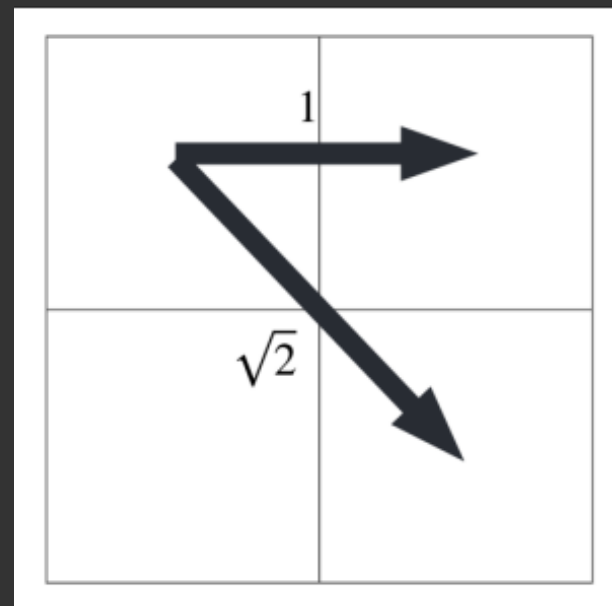
Importation d'image avec la bibliothèque  
cv2

# Parcours en largeur (BFS) :



Coût du chemin : 12.000

- Modifier les autres cas de pas





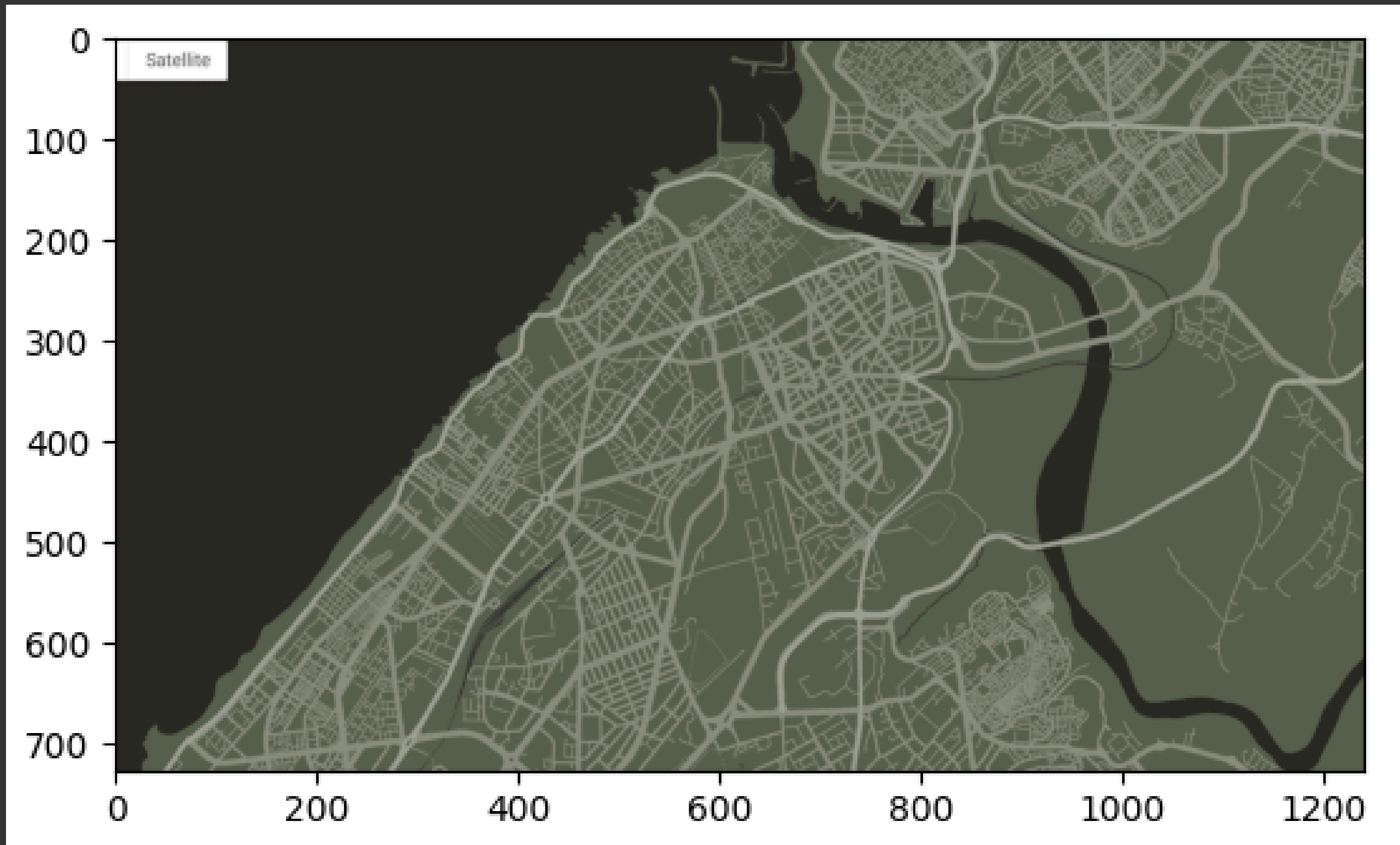
## Résultats numériques

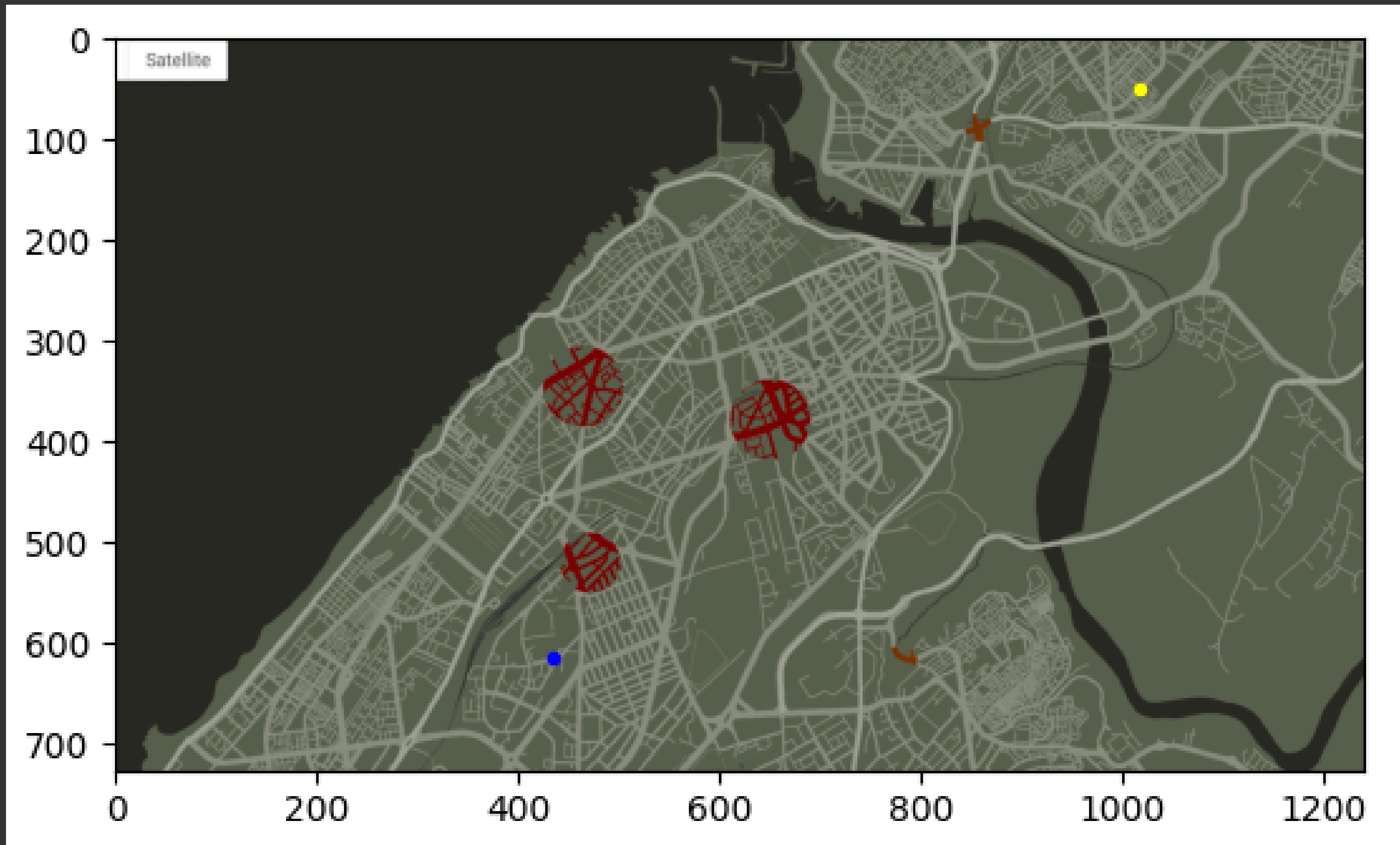
| algorithme                | BFS                | Dijkstra                     | A*                           |
|---------------------------|--------------------|------------------------------|------------------------------|
| Coût du chemin            | 22.727             | 21.899                       | 21.899                       |
| temps en secondes         | 0.0252             | 0.1003                       | 0.065                        |
| complexité (pire des cas) | $\mathcal{O}(V+E)$ | $\mathcal{O}((V+E) \log(V))$ | $\mathcal{O}((V+E) \log(V))$ |

V : le nombre des sommets

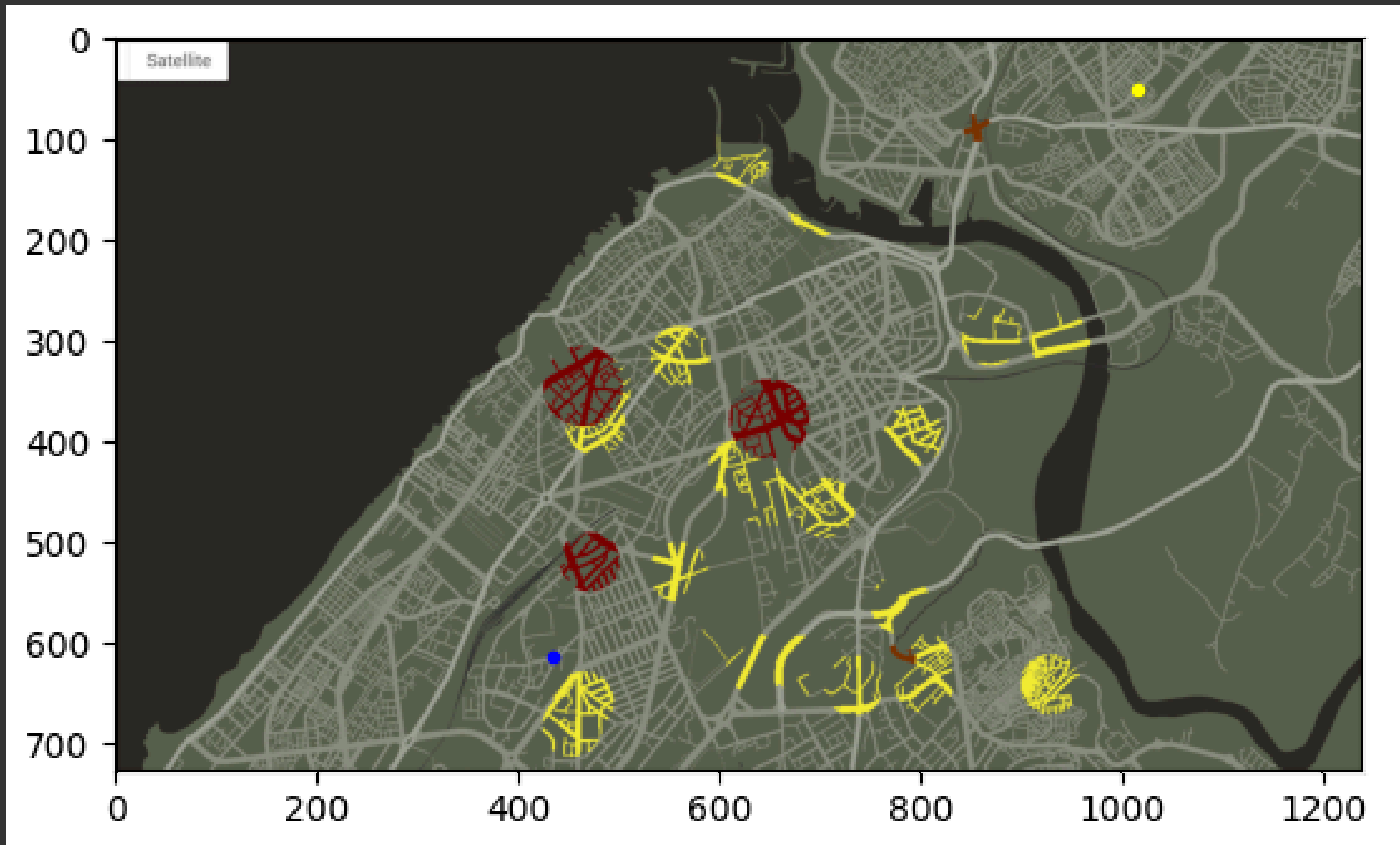
E : le nombre des arêtes

Preuve des complexités en annexe











## Résultats numériques

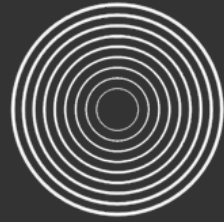
| algorithme                | BFS                | Dijkstra                     | A*                           |
|---------------------------|--------------------|------------------------------|------------------------------|
| Coût du chemin            | 1083.236           | 1042.116                     | 1042.116                     |
| temps en secondes         | 0.6397             | 4.4749                       | 2.6946                       |
| complexité (pire des cas) | $\mathcal{O}(V+E)$ | $\mathcal{O}((V+E) \log(V))$ | $\mathcal{O}((V+E) \log(V))$ |
| Optimalité du chemin      | oui                | oui                          | oui                          |

V : le nombre des sommets

E : le nombre des arêtes

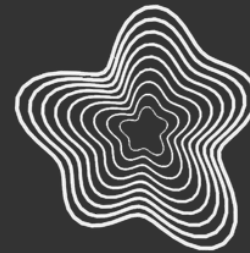
Preuve des complexités en annexe

# Remarques :



## Parcours en largeur :

- Exploration en largeur d'abord.
- Utilise une file (queue) pour gérer les nœuds à explorer.
- Parcourt tous les nœuds du même niveau avant de passer aux niveaux suivants.
- Trouve le chemin le plus court en termes de nombre d'arêtes parcourues.
- Ne prend pas en compte les poids ou les coûts des arêtes.



## Dijkstra :

- Exploration guidée par les distances.
- Utilise une file de priorité pour sélectionner le nœud avec la plus petite distance.
- Parcourt les nœuds en fonction de leurs distances actuelles à partir du nœud de départ.
- Calcule les distances les plus courtes en tenant compte des poids des arêtes.
- Peut être utilisé pour trouver le plus court chemin dans un graphe pondéré positif.



## $A^*$ :

- Exploration guidée par une fonction heuristique.
- Utilise une file de priorité pour sélectionner le nœud avec le coût total le plus bas.
- Combine les coûts réels et une estimation heuristique pour guider la recherche.
- Calcule les chemins optimaux en fonction de la fonction heuristique.
- Peut être utilisé pour trouver le plus court chemin dans un graphe pondéré positif en utilisant une estimation heuristique admissible.